Can a small local agent redo a cloud agent's work?

93 evaluator-verified tasks from an agent-built health-data repository, with an open harness and a Gemma 4 E4B baseline

John Bowyer, Health Transformation Lab

Gemma 4 Developer Agent Competition, Paper Track [17]. Topic: Tasks & Benchmarks (resource paper). November 2026.

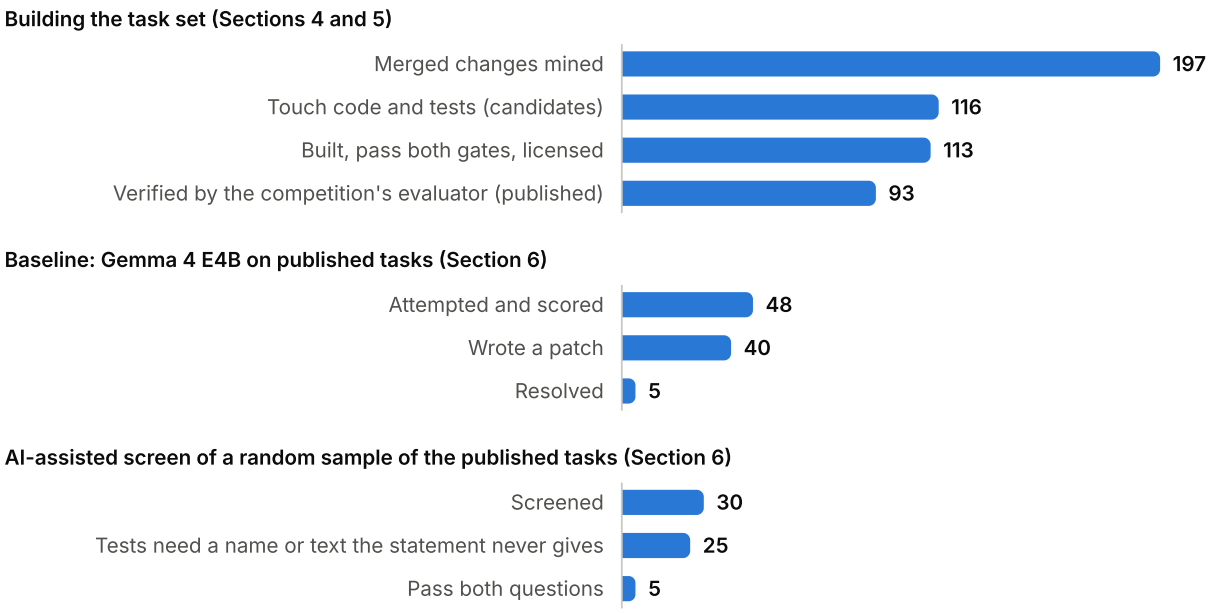
A personal learning project, not affiliated with or endorsed by any employer, institution or vendor. Every patient record in the source repository is synthetic.

ABSTRACT

Coding-agent benchmarks are mined from mature web libraries maintained by people. I release a task set mined from a different kind of repository: a health-data benchmark built by cloud coding-agent sessions under written rules that CI enforces (960 commits in fifteen days, 775 of them marked as agent-authored). It has 93 tasks in the competition's format, each verified with the competition's own evaluator and labelled with where its problem statement came from: an issue, a pull request or a commit message. Under tight limits (2,048 output tokens, thinking off), Gemma 4 E4B on two free T4 GPUs resolved 5 of 48 tasks, run cheapest first. A paired rerun of 23 with the code-graph tools resolved 2, as without them; the model rarely called the tools. An AI-assisted screen of 30 finds 25 with tests needing names the statement never gives. The agent ran on my open re-implementation of the competition's harness; the official evaluator, which did not run the agent, gives the same verdict on all 48 patches compared. I report how the model fails and what each task costs.

Data and code. Tasks (CC BY 4.0): kaggle.com/datasets/jcblearning/health-transformation-lab-coding-agent-tasks.
Harness, builders and run tables (Apache 2.0): github.com/healthtransformationlab/health-transformation-lab-nlp.
Agent configuration and each number's source (Apache 2.0): github.com/healthtransformationlab/gemma-developer-agent.

Figure 1. Task counts at each step, on one scale.



1. Introduction

The best coding agents today run on large models in the cloud. This competition [17] asks whether a model that fits on one accelerator can do the same work. The tasks available come from a narrow place: SWE-bench draws on twelve popular Python libraries [1], and the competition's development set on four.

Two kinds of repository are scarce.

The first is the **data repository**. In a web library a bug usually announces itself: a request fails, a test goes red. In health data the costly defect is silent. A pipeline finishes, a table is written, and a number in it is wrong (Section 7).

The second is the **agent-built repository**, written by agents under a person's direction. Such a repository can leave an unusually complete record: an issue for each change, a pull request, a review. A large cloud agent did this work. How much of it can a small local agent redo from the same problem statements?

I study one repository that is both. `health-transformation-lab-nlp` is part of Health Transformation Lab, open benchmarks I built on one premise: better health starts with accurate health information. Its job is to surface silent defects, here in how well gum disease can be found in synthetic dental records. I typed no code: every change was made by coding-agent sessions I directed in chat and merged after an automated review.

Contributions:

1. **A task set** of 93 tasks in the competition's format, each verified with the competition's own evaluator, public on Kaggle under CC BY 4.0, with snapshots, code graphs, embeddings and a fidelity tier per task (Section 4).
2. **An open harness checked against the official one**: a sandboxed re-implementation of the competition's documented harness that agrees with the competition's evaluator on every saved agent patch and keeps answers away from the model on shared GPUs (Section 5).
3. **A baseline** for Gemma 4 E4B with a failure breakdown, cost per task and a paired test of the code-graph tools (Section 6).
4. **A check on a second repository**: without the tracing rule, the same mining rule finds candidates but no issue-linked statements (Section 4.5).

2. Related work

Task sets from repository history. SWE-bench pairs GitHub issues with the pull requests that closed them and scores a patch by the repository's tests [1]; SWE-bench Verified adds human screening of task quality [2]. Defects4J [3] and BugsInPy [4] did the same for Java and Python bugs before language models. SWE-Gym [5], SWE-smith [6] and SWE-rebench [7] scale task collection for training and for contamination-free evaluation. All draw on human-maintained open-source libraries. This one follows the same recipe on an agent-built data repository, and labels each statement's source: issue, pull request or commit message.

Agents. SWE-agent showed that the interface given to a model matters as much as the model [8]; the competition fixes that interface, so this work re-implements it.

Data work. DS-1000 tests data-science code generation [9]; Spider 2.0 [10], ELT-Bench [11] and data-eng-bench [16] score an artefact agents produce: SQL, a pipeline, dbt models. MLOps-Bench [15] is closest: 212 tasks with fail-to-pass tests from 31 public ML repositories. These tasks score repairs to one tested, agent-built health-data codebase.

Health-data quality. The repository's checks follow the harmonized data-quality terms of Kahn et al. [12]; its de-identification tests follow the i2b2/UTHealth categories [13] and use MACCROBAT [14].

3. The source repository

The repository holds a Python package, its tests, an R twin kept in numerical parity, SQL and Databricks pipelines, and documentation. It was written between October 25 and November 8, 2026.

Who wrote it. Of 960 commits on `main`, 775 are marked as agent-authored, 175 are merge commits GitHub made for pull requests, 5 come from an automation bot, 4 carry only my name, and one is the initial commit.

Rules the agents work under. A checked-in `AGENTS.md` requires one issue per user story, a branch named for its issue, a pull request that closes it, and a test written before the code. CI rejects a pull request from an agent's own session branch or without an issue link.

Reviews. Every pull request has a review committed as a file: 240 of them. Of the 192 with a structured verdict, all approve (64 with minor findings, 9 with conditions); blocking findings are fixed on the branch first. Agents merge their own green pull requests except in four cases reserved for me: limited-compute runs, publishing outside the repository, deletions, and rule changes.

Where my time went. By my own estimate, more than half of my time went into three things a person reads: a data-model diagram annotated per attribute, a results view that drills from sensitivity, specificity, PPV, NPV and AUC down to SHAP values, and a data-quality page. How a discrepancy is displayed and drilled into decides whether anyone catches it, and no test checks that.

4. Building the task set

4.1 Mining

The mining script reads every merged pull request and every first-parent commit on `main` as of October 29, 2026: 197 changes. A change is a **candidate** when its diff touches both non-test code and test files, which is the competition's own mining rule.

Step	Tasks
Merged changes	197
Touch code and tests (candidates)	116
Built (one withdrawn: its patch names an organisation the repository's rules forbid)	115
Pass both gates	114
One withheld (its patch adds third-party files with no stated licence)	113
Published: the competition's evaluator resolves the reference patch (Section 5)	93

4.2 Fidelity tiers

Each task records where its `problem_statement` came from.

Tier	Statement source	Candidates (published)
A	the issue body	49 (39)
B	the pull request's title and body (no issue existed)	45 (34)
C	the commit message (no pull request existed)	22 (20)

Tiers B and C exist because the issue rule was adopted partway through the history. For those 67 candidates a **backfilled issue record** quotes the original text verbatim, carries a `backfilled` label and is closed on creation.

4.3 Snapshots, graphs, embeddings

A snapshot is the tree at `base_commit` as a single commit, so no later history comes along. The code graph is an AST walk over the package into NetworkX node-link JSON with the competition's keys (median 488 nodes and 397 `calls` edges). Embeddings come from EmbeddingGemma, cut to 256 dimensions and renormalised. The competition does not name its embedding model, so similarity to its vectors is unverified. A format check against the competition's own files matched on the snapshot layout, the graph keys and the embedding shape.

4.4 Gates

A task is kept only if its tests fail at `base_commit` with the test patch alone, and pass once the reference patch is applied with the files the grader strips removed. All 115 fail without the fix; 114 pass with it.

4.5 A second repository

To see whether the recipe transfers, I applied the mining rule unchanged to a second public repository: Open Navigator, run by a nonprofit I lead, with several human contributors and no issue-tracing rule. Of 1,320 first-parent commits, 133 touch non-test Python and Python tests. None links an issue. Only 22 can be tied by commit subject to one of the repository's 169 merged pull requests.

So the rule finds candidates anywhere, but statements depend on what the history recorded: 67 of this paper's 116 candidates got their issue record afterwards. These 133 are not gated and not released.

5. Harness and verification

My harness. The competition's guide documents the initial prompt, the multi-turn loop, the nine tools with their limits, the budgets, patch extraction and verification. I re-implemented those sections before I had the competition's evaluator: it is not on PyPI or among the competition's files, and I missed the separate Kaggle dataset that carries it. The prompt, nudge messages and budget warning follow the guide's wording. Two scripted models bound the harness. Replaying each reference patch from the repository's history resolves 111 of 113 tasks (the other two need Postgres and R), and submitting nothing resolves none.

The competition's evaluator. I then ran the competition's own verification (`swegemma` 0.2.11, subprocess sandbox, unmodified) on the 113 tasks first published and on the corrected release.

Check	Result
First release as published, empty patch (3 tasks)	all 3 fail at import, before any test runs
One file added per snapshot, reference patch	93 of 113 resolved
One file added per snapshot, empty patch	0 of 113 resolved
One file added per snapshot, Gemma's saved patches	same verdict as my harness on 25 of 25
Corrected release as published, reference patch	93 of 93 resolved
Corrected release as published, empty patch	0 of 93 resolved
Corrected release as published, Gemma's rerun patches	same verdict as my harness on 23 of 23

The first release could not be scored: the evaluator puts the repository root on the import path, and this package sits in `python/`. The corrected release adds a short `conftest.py` at each snapshot's root and keeps only the 93.

The 20 left out fall into three groups. Twelve have tests that read third-party files the published snapshots withhold for licence reasons, so they cannot pass from a snapshot under any verifier; my control missed this because it ran from the repository's history. Six start a script that imports a module beside it, which the evaluator's `PYTHONSAFEPATH=1` forbids and my harness allows. Two need Postgres or R. Elsewhere the two verifiers agree on every reference patch.

Keeping answers from the model. This repository's history contains every fix, and later snapshots contain earlier tasks' fixes. Commands and verification run in Linux user, mount, network and pid namespaces with no network and an allow-listed environment. A review before the first pull request found that an earlier version ran the model's commands on the host, where it could read every reference patch.

Kaggle GPUs do not allow namespaces, and a notebook can read every attached file. So each task runs alone in a private, offline notebook beside a private dataset holding that task only. The patch comes back and is verified locally.

Known differences. Namespaces instead of a container; no ADK context compaction or prefix caching; subagents and skills are not run; `PYTHONSAFEPATH` is not set. No number here is a competition score.

6. Baseline

Setup. `gemma-4-e4b-it` in fp16 with transformers on two T4 GPUs, internet off, thinking off, at most 2,048 new tokens per turn, a 24,576-token context, 40 tool calls, 60 turns and 30 minutes per task. Generation runs at about 8 tokens a second, hence 30 minutes. Tasks were run cheapest first under a GPU-hour cap.

Results. Ten of the 58 tasks attempted are among the twenty left out (Section 5), so they are set aside. On the other 48 the model wrote a patch for 40 and resolved 5 (10.4%, 95% Wilson CI 4.5–22.2). By tier, it resolved 4 of 24 tasks whose statement is an issue, 1 of 19 from a pull request and 0 of 5 from a commit message. The intervals overlap.

Do the graph tools help? A fault in my harness served the first tasks without their code graph and embeddings, so the three graph tools had nothing to query. After fixing it I reran 28 of them with the same settings; the 23 published give a paired comparison.

23 tasks, each run twice	Non-empty patch	Resolved (95% CI)
Graph tools unavailable	18	2 (8.7%, 2.4–26.8)
Graph tools available	20	2 (8.7%, 2.4–26.8)

One task was resolved only without the tools, one only with them, one both times. The transcripts suggest why: with the tools available the model called one in 4 of the 23 runs, 6 calls among 145. It reads and edits files and rarely searches. The figures above and below use each task's latest run; eight tasks have none with the tools.

How it fails. Of the 43 unresolved tasks, 35 submitted a patch that fails the target tests (30 with failing tests, 5 with errors before any test ran) and 8 submitted nothing. In those eight transcripts, four have a turn that hit the 2,048-token limit, three submit without an edit, and one calls the edit tool four times with an argument it does not take. The five resolved tasks are small: a CI rule, a warning message, a label parser, a training setting, a notebook setting.

Task fairness. An AI reviewer screened 30 of the 93 on SWE-bench Verified's two questions [2]; I checked its findings in summary and agreed with all 30. Five pass. In 25 the tests need a name or text the statement never gives. Of 16 the model attempted, it resolved 1 of the 3 that pass and none of the 13 that fail, so the baseline may measure the tasks more than the model.

Cost. The 58 first attempts used 20.6 GPU-hours and the 28 reruns 9.6: 30.2 in all, about $21 at an estimated $0.69 per T4-hour; Kaggle's GPUs are free. The median task takes 0.39 GPU-hours, 7 tool calls and about 91,000 prompt tokens. Five resolved tasks cost 6.0 GPU-hours each.

7. What these tasks contain that web-library tasks do not

Three cases from the repository show the defect class that motivates it: everything ran, and a number was wrong.

Case	Score on what it was built against	Score on what it was not
De-identification rules, share of identifiers removed	1.00 (the tier the rules were written beside)	0.27 (a tier written after the rules were frozen)
Tooth-loss model, AUC	0.89–0.90 (held-out labelled extractions)	0.59–0.63 (teeth missing at the first visit)
A masking defect in the feature table	every test passed	the measures of about a quarter of charted teeth were blank in every earlier run

8. Limitations

- **One repository, one author's rules.** The second repository tests only the mining rule.
- **Agent-written statements.** The issue text was largely written by a cloud agent at my direction. That may make them easier or harder to follow than a person's.
- **My harness ran the agent.** The competition's evaluator verified patches afterwards but did not run the agent, so no score here is comparable to the leaderboard.
- **Constrained settings.** The baseline caps output at 2,048 tokens with thinking off on T4 GPUs. My competition configuration allows 16,384 output tokens and a 4,096-token thinking budget on four L4s. It is a constrained baseline, not an estimate of the model under the competition's budgets.

- **Small.** 48 of 93 tasks, cheapest first, one sampled run per condition at temperature 0.2. One task flipping moves the paired result.
- **A screen checked only in summary.** I did not read the tests; the AI's verdict was shown first. 63 tasks are unscreened.
- **Untested components.** The submission's analyzer sub-agent and its skill were not run.
- **The tasks are not silent.** Each comes with a failing test. The set measures repair of the code that looks for silent defects, not noticing one unprompted.
- **Synthetic health data.** The cases in Section 7 are agreement with a synthetic answer key, not findings about any patient or practice.

9. Conclusion and release

A cloud agent built this repository in fifteen days. From the same problem statements, a model that runs on free GPUs resolved 5 of 48 tasks, all of them small, and left the graph tools almost unused. The released tasks let others test other models and settings.

The recipe should transfer to any repository that traces its changes to issues: mine the changes that touch code and tests, record where each statement came from, gate, and run a reference-patch control and an empty-patch control on the published files, with the official evaluator where there is one, then screen statements against tests, before trusting any model's score.

References

1. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR 2024.
2. Chowdhury et al. Introducing SWE-bench Verified. OpenAI, 2024.
3. Just, Jalali, Ernst. Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs. ISSTA 2014.
4. Widyasari et al. BugsInPy: A Database of Existing Bugs in Python Programs. ESEC/FSE 2020.
5. Pan et al. Training Software Engineering Agents and Verifiers with SWE-Gym. ICML 2025.
6. Yang et al. SWE-smith: Scaling Data for Software Engineering Agents. NeurIPS 2025.
7. Badertdinov et al. SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents. NeurIPS 2025.
8. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. NeurIPS 2024.
9. Lai et al. DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation. ICML 2023.
10. Lei et al. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. ICLR 2025.
11. Jin, Zhu, Kang. ELT-Bench: An End-to-End Benchmark for Evaluating AI Agents on ELT Pipelines. PVLDB, 2026.
12. Kahn et al. A Harmonized Data Quality Assessment Terminology and Framework for the Secondary Use of Electronic Health Record Data. eGEMs, 2016.
13. Stubbs, Uzuner. Annotating longitudinal clinical narratives for de-identification: the 2014 i2b2/UTHealth corpus. Journal of Biomedical Informatics, 2015.
14. Caufield et al. A Comprehensive Typing System for Information Extraction from Clinical Narratives (MACCROBAT). 2019.
15. Tran, Patel, Mahajan et al. MLOps-Bench: Benchmarking AI Agents on Production ML Engineering in Real Repositories. arXiv:2610.23506, 2026.
16. Snowflake Inc. data-eng-bench: a data-engineering benchmark for coding agents. Version 1.0.0, github.com/Snowflake-Labs/data-eng-bench, 2026.
17. Markowitz, Perozzi, Rózemberczki, Cameron, Hemmati, Li, Galkin, Farhadi, Holbrook, Oldacre. Google - The Gemma 4 Developer Agent Competition. Kaggle, 2026. https://www.kaggle.com/competitions/gemma-4-developer-agent